

Abstract

Design, implement, test, simulate real services, log attack attempts, analyze traffic by demonstrating a basic network security 'honeypot'. Setting up a basic five network services, probing them from attacking machines, capturing traffic, and analyzing logs and results through network intercepting tools.

Introduction

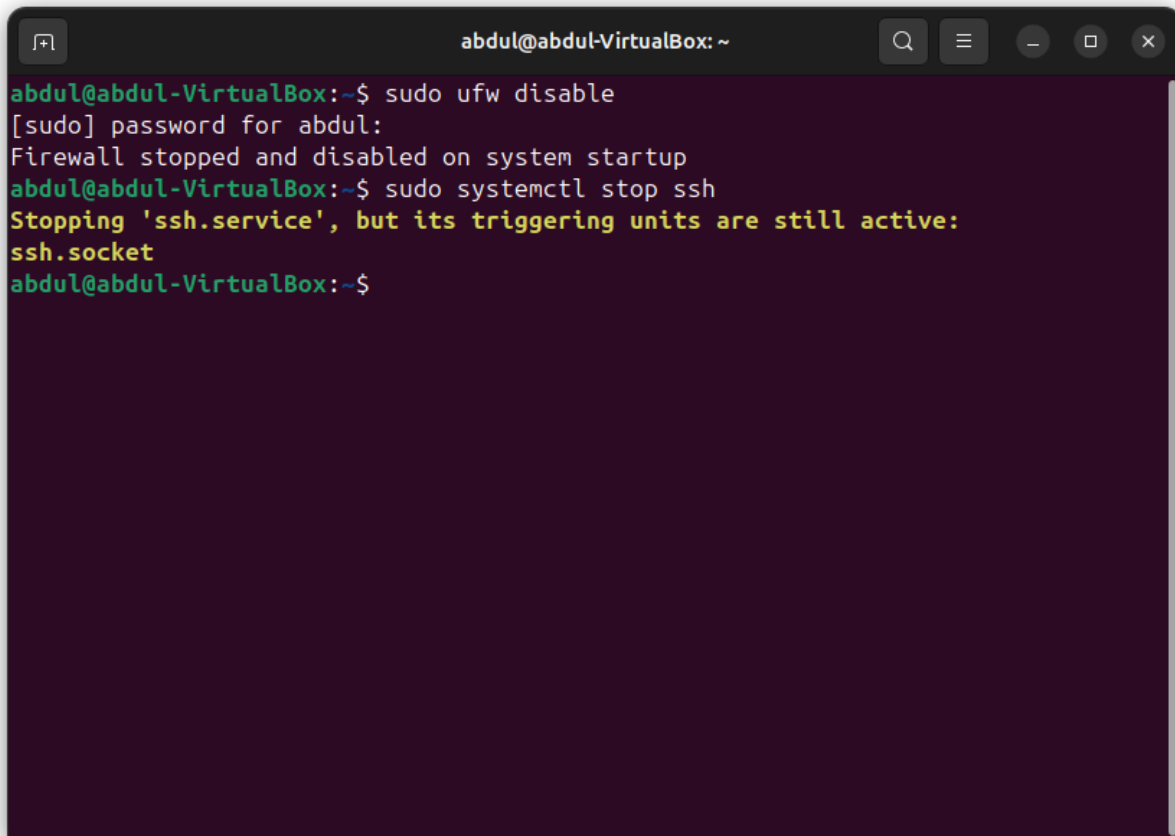
Using Virtualbox to run two different operating systems as Virtual machines. Using Kali Linux virtual machine and an Ubuntu virtual machine for this project. Kali will be the attacking system. Ubuntu will be the target system. Networks are being set on Bridged adapter mode. Netcat will be used to send and receive unencrypted data. Wireshark will be used for packet analysis. SSH "Secure Shell" is used for communication between these two systems on port 2222. Nmap and Zenmap aGUI of Nmap, Telnet on (Port 23), FTP (Port 21), NetCat is used to respond with an FTP banner when a connection is made, and it logs the details. SMTP (Port 25): using Netcat to simulate an SMTP server by sending a response banner and logging the connection. HTTP (Port 80), A simple Python HTTP server is used to simulate a web service. It listens for incoming HTTP requests and responds with basic content.

-All commands used are written in a separate file attached to the project.

Summary of Results

Now run both Kali Linux and Ubuntu Vm's and before starting both Vm's, change the network to Bridged Adapter mode and connect both Vm's to the Network. And also turn on Promiscuous mode. Now go to Ubuntu Machine and open terminal and type the command "sudo ufw disable" this command will turn off the Uncomplicated Firewall (UFW) on our Linux system, stopping all active firewall rules from filtering network traffic. As a result, any restrictions previously set such as blocked ports, allowed IP addresses, or denied services are no longer enforced, and your system becomes fully open to incoming and outgoing connections. This can make it easier to test network tools or troubleshoot connectivity issues. After that stop ssh service to set up ssh honeypot on another port using the command "sudo systemctl stop ssh" this will make

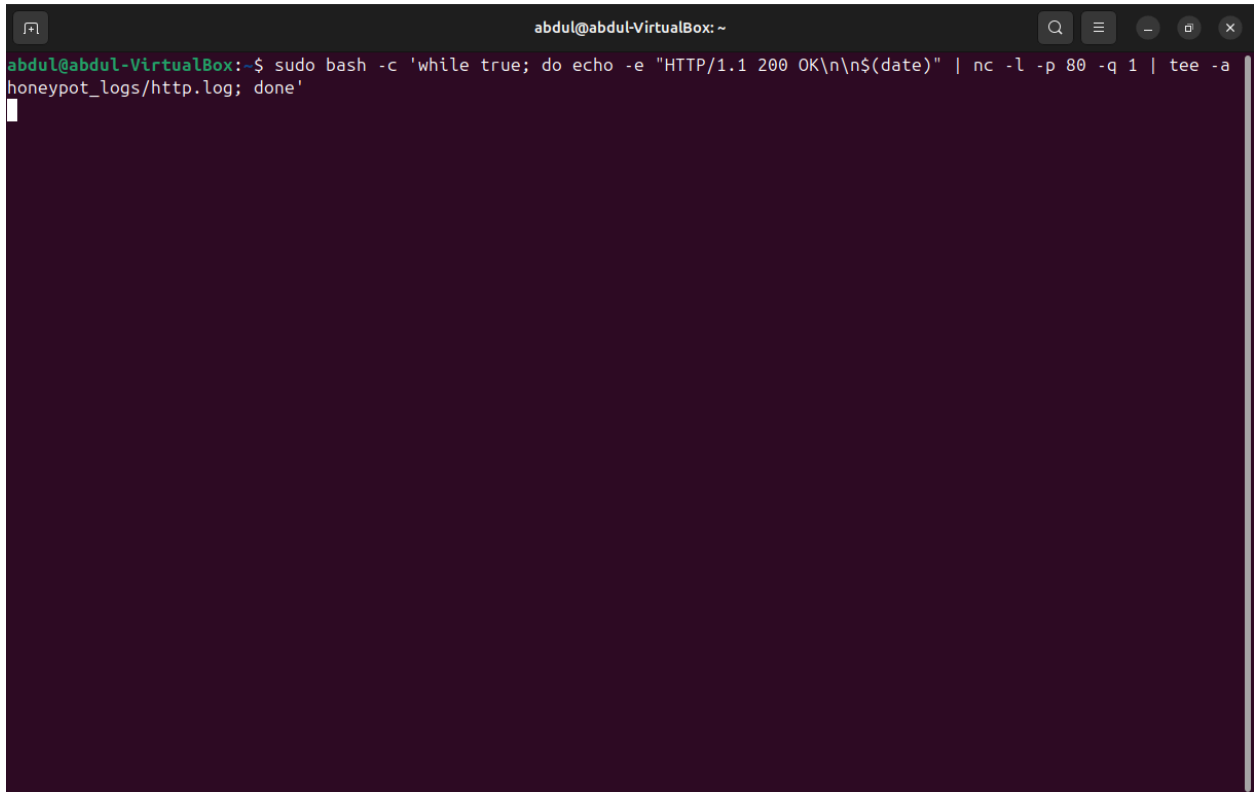
the ssh inactive. Now create a directory using the command “mkdir -p honeypot_logs”

A terminal window titled 'abdul@abdul-VirtualBox: ~' with standard window controls. The terminal shows the following commands and output:

```
abdul@abdul-VirtualBox:~$ sudo ufw disable
[sudo] password for abdul:
Firewall stopped and disabled on system startup
abdul@abdul-VirtualBox:~$ sudo systemctl stop ssh
Stopping 'ssh.service', but its triggering units are still active:
ssh.socket
abdul@abdul-VirtualBox:~$
```

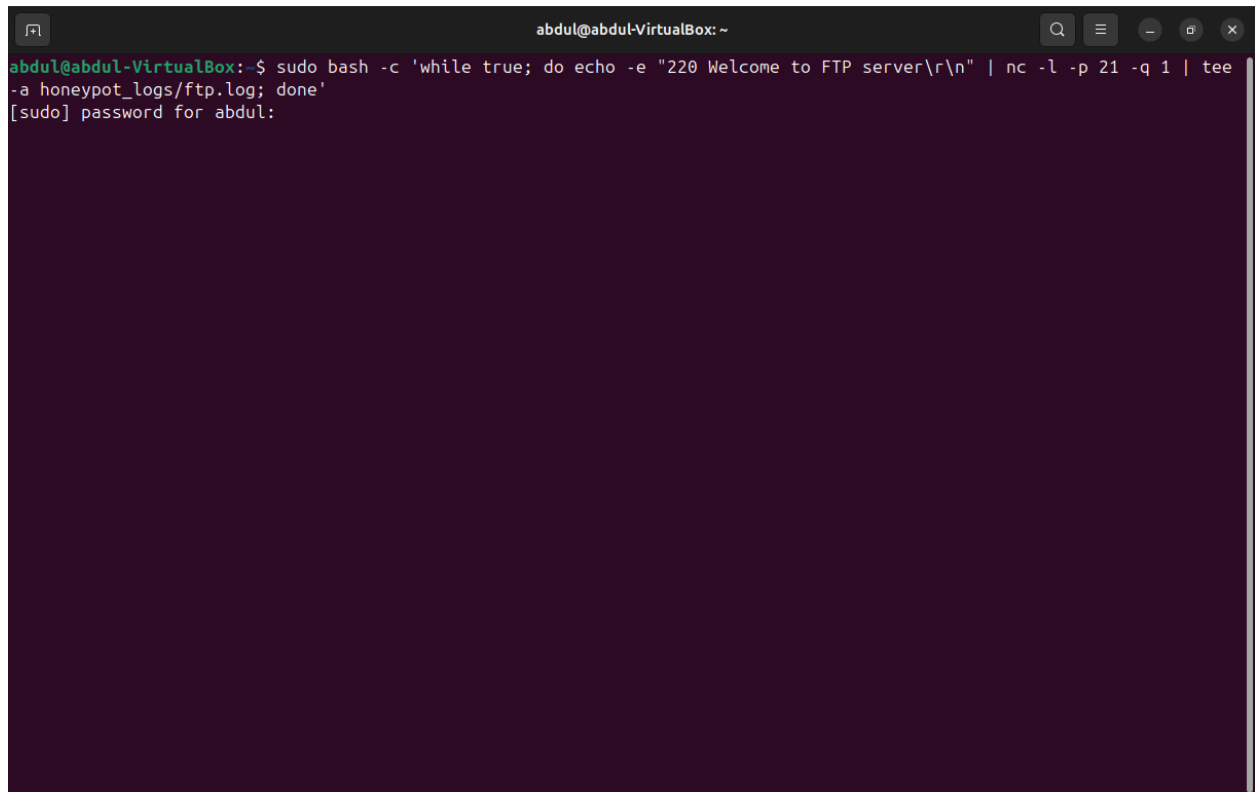
This will create a folder called honeypot_logs in the Ubuntu machine. Now we have to create 5 different network services to simulate as a honeypot. First we are going to create a http server using the netcat, a versatile networking tool used for reading from and writing to network connections using TCP or UDP. It's commonly used for debugging, network exploration, and in security testing for tasks like setting up servers, testing connections, or even transferring files. To create the http server we use the command “sudo bash -c 'while true; do echo -e "HTTP/1.1 200 OK\n\n\$(date)" | nc -l -p 80 -q 1 | tee -a honeypot_logs/http.log; done” This command sets up a basic HTTP server using Netcat, designed to simulate a service for a honeypot. It listens for incoming connections on port 80 and responds with a simple HTTP 200 OK status along with the current date and time. The server runs in an infinite loop, continually accepting and responding to requests. The response, including the timestamp, is both printed to the console and logged to a file (honeypot_logs/http.log) for later analysis. The logging functionality allows the capture of incoming requests, helping monitor any

interactions or potential attacks targeting the simulated HTTP service.

A terminal window titled 'abdul@abdul-VirtualBox: ~' with standard window controls. The command prompt shows the execution of a command to start an HTTP honeypot: `abdul@abdul-VirtualBox:~$ sudo bash -c 'while true; do echo -e "HTTP/1.1 200 OK\n\n$(date)" | nc -l -p 80 -q 1 | tee -a honeypot_logs/http.log; done'`. The command is partially executed, with a cursor visible at the end of the line.

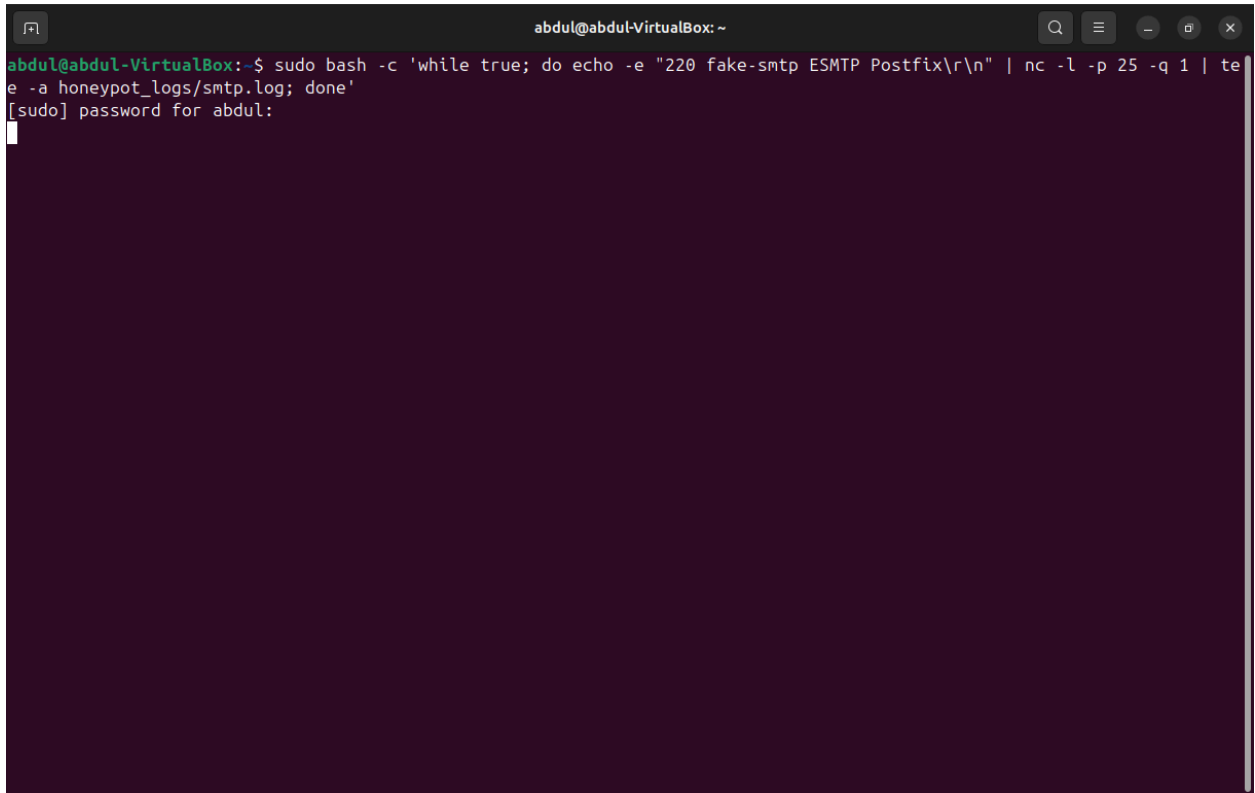
```
abdul@abdul-VirtualBox:~$ sudo bash -c 'while true; do echo -e "HTTP/1.1 200 OK\n\n$(date)" | nc -l -p 80 -q 1 | tee -a
honeypot_logs/http.log; done'
```

Then we are going to set up a FTP (File Transfer Protocol) server honeypot. An FTP server is a service that allows users to transfer files over a network using the FTP protocol. FTP servers are commonly used for uploading and downloading files, especially for tasks like web development, file sharing, and remote backups. To setup Ftp server we are going to use NetCat and the command would be “`sudo bash -c 'while true; do echo -e "220 Welcome to FTP server\r\n" | nc -l -p 21 -q 1 | tee -a honeypot_logs/ftp.log; done'`” This command sets up a basic FTP server using Netcat on port 21, simulating a simple FTP service for a honeypot. It listens for incoming connections and responds with a standard FTP greeting message (220 Welcome to FTP server). The server runs in an infinite loop, continuously accepting connections and sending the greeting message to any client that connects. Each interaction is logged to the `honeypot_logs/ftp.log` file, allowing the administrator to track connection attempts. This setup is useful for monitoring and analyzing potential malicious activity targeting an FTP service, as attackers may attempt to interact with the simulated server. The server is intentionally minimal, only responding with the greeting, making it an effective tool for detecting probing or exploitation attempts.

A terminal window titled 'abdul@abdul-VirtualBox: ~' with search, menu, and window control icons in the title bar. The terminal shows a user running a command to set up an FTP honeypot. The command is: `sudo bash -c 'while true; do echo -e "220 Welcome to FTP server\r\n" | nc -l -p 21 -q 1 | tee -a honeypot_logs/ftp.log; done'`. The prompt changes to `[sudo]` and the user is asked for a password, which is entered as 'abdul'.

Once we set up our ftp server we are going to set up SMTP (Simple Mail Transfer Protocol), it is the standard protocol used for sending emails over the internet. It is a text-based, push protocol that allows the transmission of email from a sender's email client or server to a recipient's email server. SMTP operates mainly for sending messages, while other protocols like POP3 (Post Office Protocol) or IMAP (Internet Message Access Protocol) are used for retrieving and managing messages. To set up this we are going to use the command "sudo bash -c 'while true; do echo -e "220 fake-smtp ESMTP Postfix\r\n" | nc -l -p 25 -q 1 | tee -a honeypot_logs/smtp.log; done'" This command sets up a fake SMTP server using Netcat on port 25, simulating a Postfix mail server that responds to incoming client connections with a standard SMTP greeting message (220 fake-smtp ESMTP Postfix). The server operates in an infinite loop, continuously listening for connection attempts, and each time a client connects, it sends a response indicating that the server is ready to accept mail. The message "220 fake-smtp ESMTP Postfix" mimics a typical SMTP server greeting, suggesting the presence of a real server. Every interaction with the server, including incoming connections and responses, is logged to honeypot_logs/smtp.log, allowing administrators to monitor and analyze these interactions. By simulating a real SMTP service, the honeypot helps to detect unauthorized access attempts, identify potential attackers, and gather intelligence on the types of probing or exploitation techniques

used. The log file provides valuable insights into who is attempting to interact with the server, enabling further analysis of the network traffic and improving overall security monitoring.

A terminal window titled 'abdul@abdul-VirtualBox: ~' with standard window controls. The command entered is 'sudo bash -c 'while true; do echo -e "220 fake-smtp ESMTP Postfix\r\n" | nc -l -p 25 -q 1 | tee -a honeypot_logs/smtp.log; done''. The prompt '[sudo] password for abdul:' is visible, and the cursor is on the next line.

```
abdul@abdul-VirtualBox: ~  
abdul@abdul-VirtualBox:~$ sudo bash -c 'while true; do echo -e "220 fake-smtp ESMTP Postfix\r\n" | nc -l -p 25 -q 1 | tee  
e -a honeypot_logs/smtp.log; done'  
[sudo] password for abdul:  
|
```

Now, since we had set up three different honeypot services, now we are going to create two simple Python listener scripts for SSH (custom port) and Telnet. First we are going to create a SSH script. This Python script sets up a simple SSH honeypot that listens for incoming connections on port 2222 and simulates an SSH server by responding with a standard SSH banner. When a client connects, the honeypot logs the client's IP address and any data received, storing the information in a log file (ssh.log). The server continues to accept connections in an infinite loop, providing an effective way to monitor potential malicious activity targeting SSH services. The honeypot logs all interactions, which can help identify attackers attempting to brute-force SSH credentials or probe for vulnerabilities. This script serves as a basic tool for security researchers and system administrators to detect unauthorized attempts to access SSH services, providing valuable insight into attack patterns and behaviors. The honeypot can be extended to simulate more advanced SSH functionalities, making it a useful component in a broader network security monitoring strategy.

A screenshot of a code editor window. The title bar shows 'ssh.py' and standard window controls. The editor content is a Python script for an SSH honeypot. It imports socket, datetime, and os. It sets HOST to '0.0.0.0', PORT to 2222, LOG_DIR to 'honeypot_logs', LOG_FILE to 'ssh.log', and BANNER to 'SSH-2.0-OpenSSH_8.2p1 Ubuntu-4'. It ensures the log directory exists with os.makedirs. It defines a log function that writes messages to the log file with a timestamp. It defines start_ssh_honeypot which binds a socket to the host and port, listens, and enters a while loop to accept connections and log them.

```
import socket
import datetime
import os

HOST = "0.0.0.0"
PORT = 2222
LOG_DIR = "honeypot_logs"
LOG_FILE = os.path.join(LOG_DIR, "ssh.log")
BANNER = b"SSH-2.0-OpenSSH_8.2p1 Ubuntu-4\r\n"

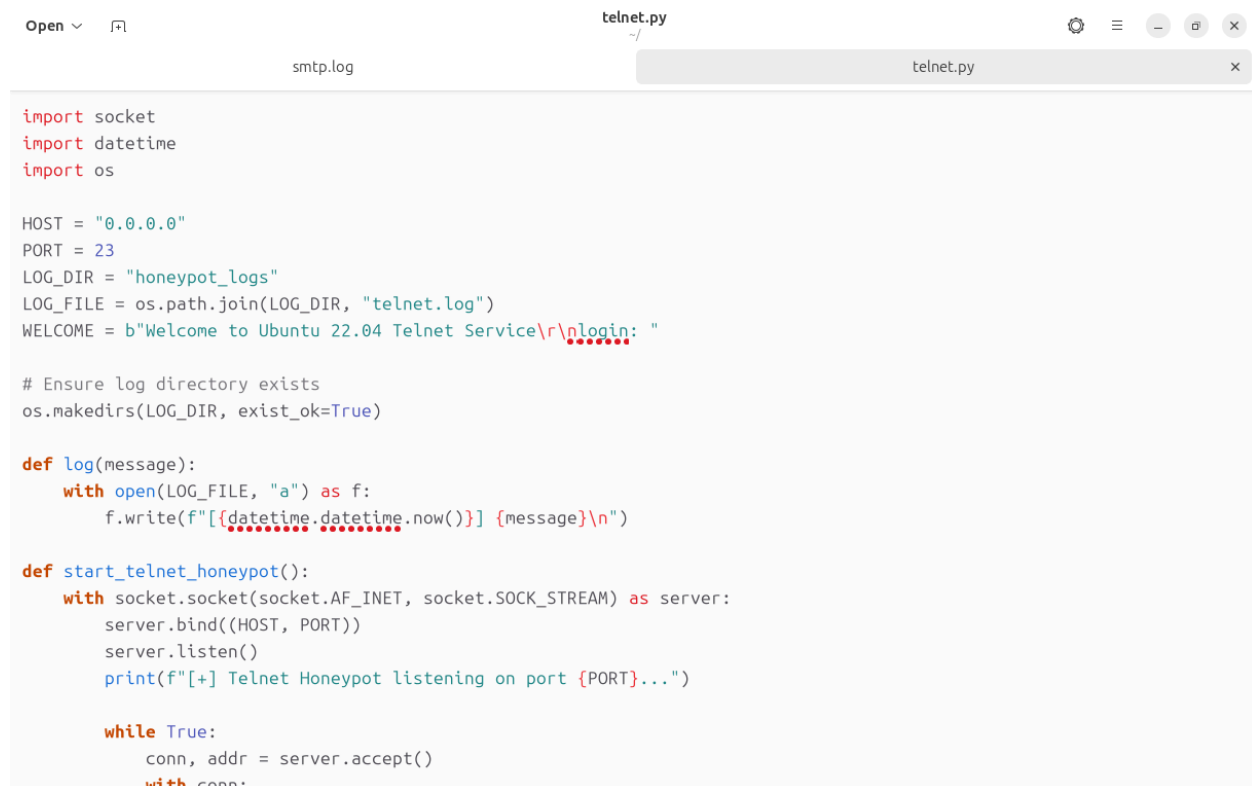
# Ensure log directory exists
os.makedirs(LOG_DIR, exist_ok=True)

def log(message):
    with open(LOG_FILE, "a") as f:
        f.write(f"[{datetime.datetime.now()}] {message}\n")

def start_ssh_honeypot():
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as server:
        server.bind((HOST, PORT))
        server.listen()
        print(f"[+] SSH Honeypot listening on port {PORT}...")

    while True:
        conn, addr = server.accept()
        with conn:
```

Now after the SSH python script we are going to create a Telnet script. This Python script sets up a simple Telnet honeypot that listens for incoming connections on port 23, which is the default port for Telnet services. When a client connects, the honeypot sends a simulated welcome message (Welcome to Ubuntu 22.04 Telnet Service\r\nlogin:) to prompt for login credentials. The script logs the incoming connection's IP address and any data received from the client into a log file (telnet.log). It runs in an infinite loop, continuously accepting connections and logging interactions, which can help identify potential attackers or unauthorized users scanning for vulnerable Telnet services. By simulating a Telnet server, the honeypot attracts malicious actors attempting to brute-force login credentials or exploit known Telnet vulnerabilities. The logging functionality captures relevant details, such as connection attempts and any data sent, which can be analyzed for signs of intrusion or attempted exploitation.

A screenshot of a code editor window titled 'telnet.py'. The editor shows a Python script for a Telnet honeypot. The script imports socket, datetime, and os. It sets HOST to '0.0.0.0', PORT to 23, LOG_DIR to 'honeypot_logs', LOG_FILE to 'telnet.log', and WELCOME to 'Welcome to Ubuntu 22.04 Telnet Service\r\nlogin: '. It ensures the log directory exists and defines a log function. The start_telnet_honeypot function binds the server to the host and port, listens, and prints a message. A while loop keeps the server running, accepting connections and logging them.

```
import socket
import datetime
import os

HOST = "0.0.0.0"
PORT = 23
LOG_DIR = "honeypot_logs"
LOG_FILE = os.path.join(LOG_DIR, "telnet.log")
WELCOME = b"Welcome to Ubuntu 22.04 Telnet Service\r\nlogin: "

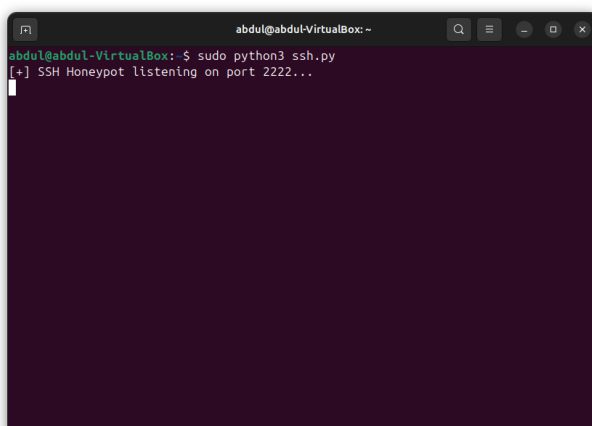
# Ensure log directory exists
os.makedirs(LOG_DIR, exist_ok=True)

def log(message):
    with open(LOG_FILE, "a") as f:
        f.write(f"[{datetime.datetime.now()}] {message}\n")

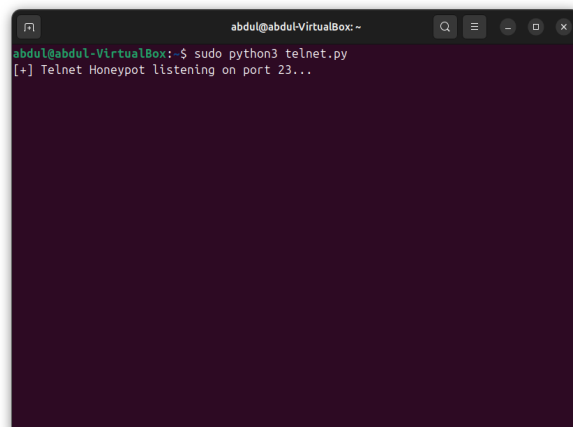
def start_telnet_honeypot():
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as server:
        server.bind((HOST, PORT))
        server.listen()
        print(f"[+] Telnet Honeypot listening on port {PORT}...")

    while True:
        conn, addr = server.accept()
        with conn:
```

Once we have both the scripts we are going to run them. To start it, we are going to run the scripts by using the command “sudo python3 [ssh.py](#)”. This will start the python script and start running the ssh Honeypot on port 2222. Now, we are going to run the scripts by using the command “sudo python3 telnet.py”. This will start the python script and start running the Telnet Honeypot on port 23.

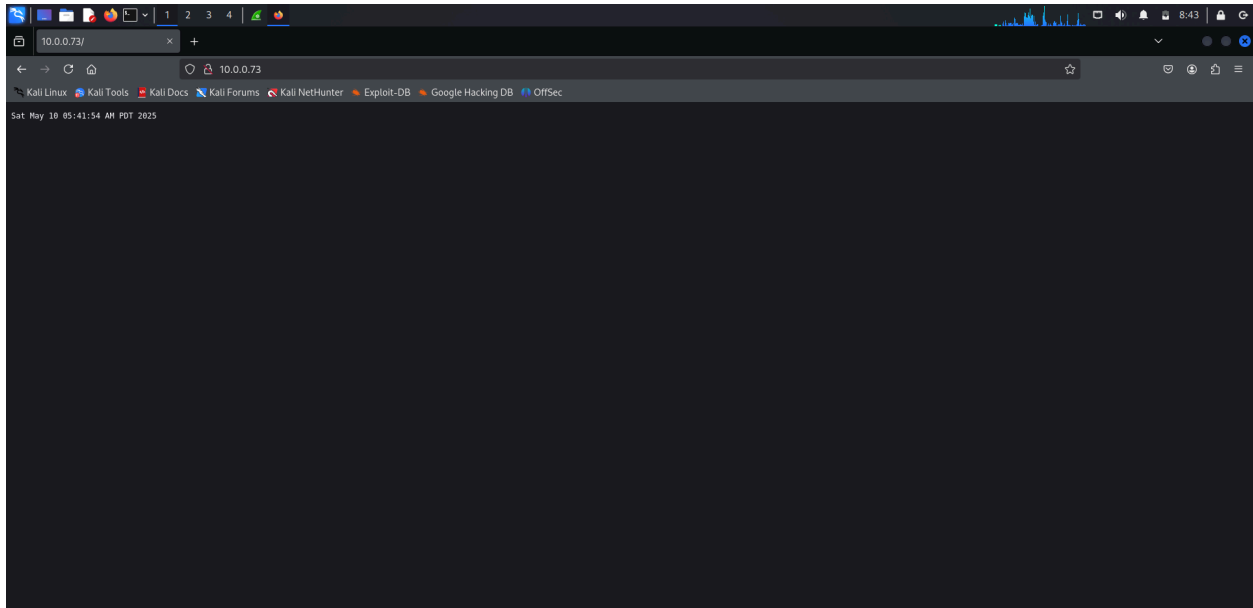
A terminal window titled 'abdul@abdul-VirtualBox: ~' showing the command 'sudo python3 ssh.py' being executed. The output is '[+] SSH Honeypot listening on port 2222...'.

```
abdul@abdul-VirtualBox: ~
abdul@abdul-VirtualBox:~$ sudo python3 ssh.py
[+] SSH Honeypot listening on port 2222...
```

A terminal window titled 'abdul@abdul-VirtualBox: ~' showing the command 'sudo python3 telnet.py' being executed. The output is '[+] Telnet Honeypot listening on port 23...'.

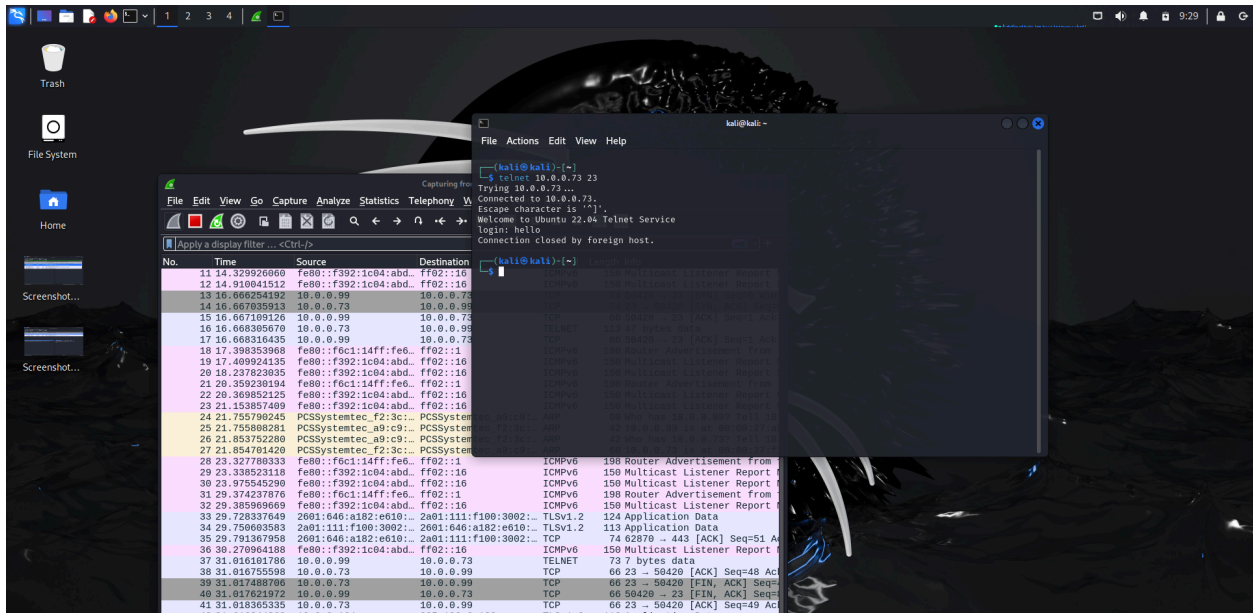
```
abdul@abdul-VirtualBox: ~
abdul@abdul-VirtualBox:~$ sudo python3 telnet.py
[+] Telnet Honeypot listening on port 23...
```

After we are going to Kali machine and going on to browser, and typing the command “<http://10.0.0.73:80>” this will access the page hosted by the Netcat terminal from ubuntu, in return it shows us a message of Date, time like as shown in the picture below.

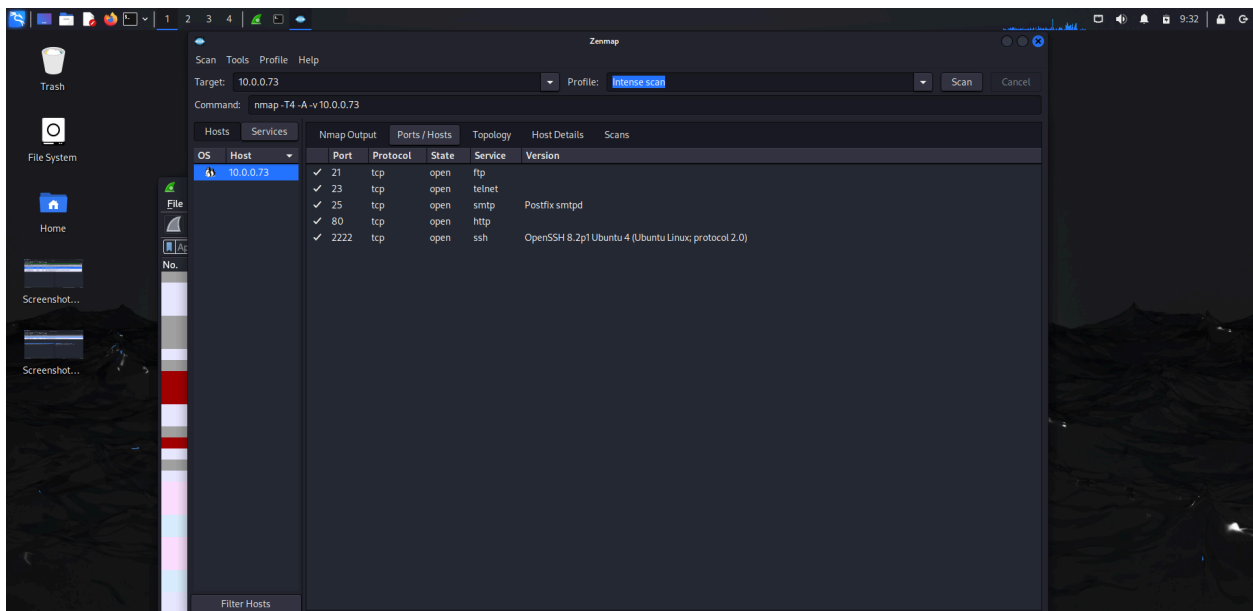


And when we go to the Ubuntu Netcat terminal, we can see the logs of our ip accessing the server.

After that we are doing a Telnet honeypot interaction and its corresponding network traffic analysis using Wireshark on Kali Linux. In the terminal, a Telnet connection is initiated to the target IP 10.0.0.73 on port 23 using the command "telnet 10.0.0.73 23" , which is hosting a fake Telnet service. The honeypot responds with a welcome banner and login prompt, mimicking a real Ubuntu Telnet server. After the user inputs "hello", the connection is deliberately closed by the honeypot, a common tactic to log attacker behavior without providing actual access. Simultaneously, Wireshark monitors the network, displaying the Telnet session packets, including SYN, ACK, and FIN flags, which indicate the establishment and termination of the TCP connection.



Now we are going to run the nmap intense scan using the command “nmap -p 1-65535 -T4 -A -v 10.0.0.73” this will start scanning and the wireshark is also running the background. And when we completed the nmap scan we found the 5 ports are open and the logs are also entered in the honeypot.



After this, when analyzing wireshark packet captures, the packets display network packets that reveal valuable details about the attacker's behavior. When unencrypted protocols like FTP, Telnet, or SMTP are targeted, you can clearly see commands, login attempts, and even credentials in plain text within the packet payloads. For example,

attackers scanning for open services or trying to log in will generate visible TCP connection attempts and possibly banner responses from the honeypot. When clearly observed tools like Nmap leave identifiable traces, such as specific probe packets or incomplete connection attempts. If encrypted services like SSH are probed, the data appears as unreadable binary streams, indicating that the communication is secured. Overall, Wireshark provides a clear, time-stamped view of the attack sequence, and the logs give us a more detailed version of that.



The image displays three screenshots of log files from a honeypot, showing various connection attempts and data received. Each screenshot is a window titled with the log file name and the path ~/honeypot_logs.

ssh.log

```
[2025-05-10 06:25:01.944960] Connection from 10.0.0.99
[2025-05-10 06:25:01.946086] Received data from 10.0.0.99: SSH-2.0-OpenSSH_9.9p1 Debian-3

[2025-05-10 06:25:45.553451] Connection from 10.0.0.99
[2025-05-10 06:25:45.553711] Received data from 10.0.0.99: SSH-2.0-OpenSSH_9.9p1 Debian-3
```

http.log

```
GET / HTTP/1.1
Host: 10.0.0.73
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/png,image/svg+xml,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Connection: keep-alive
Upgrade-Insecure-Requests: 1
Priority: u=0, i

GET / HTTP/1.1
Host: 10.0.0.73
User-Agent: curl/8.11.0
Accept: */*
```

telnet.log

```
[2025-05-10 06:29:24.175727] Connection from 10.0.0.99
[2025-05-10 06:29:38.531543] Received data from 10.0.0.99: hello
```

The above are the logs of honeypot in the files created during the attack.

Conclusion:

This project successfully demonstrated the setup and deployment of a basic honeypot on an Ubuntu machine that simulates five common network services: SSH, Telnet, FTP, SMTP, and HTTP. Using simple tools such as Netcat and Python, we created decoy services that recorded all incoming connections and logged attacker interactions. Probing the honeypot from a Kali Linux system using tools like nmap allowed us to simulate reconnaissance activity, and Wireshark provide visibility into the packet-level details of these interactions. Honeypots can significantly improve network security by acting as an early warning system. By intentionally exposing fake services, they attract unauthorized traffic, enabling defenders to detect scanning behavior and attempted exploits that might otherwise go unnoticed. This proactive approach helps identify vulnerabilities in a controlled environment without risking production systems. Additionally, honeypots can gather valuable intelligence on attacker behavior. This includes the techniques and tools used for scanning, login attempts, and protocol manipulation. For example, by analyzing login patterns or payloads, defenders can learn about brute-force strategies, common usernames/passwords, or even malware delivery attempts. Such data can inform system hardening and guide the creation of more robust intrusion detection rules. However, honeypots are passive by design and they do not block or mitigate threats. To create a more active defense system, they can be combined with firewalls and intrusion prevention systems. For example, firewall rules can be dynamically updated to block IP addresses detected interacting with the honeypot, effectively quarantining suspicious actors. This hybrid approach enhances both detection and response capabilities, turning passive monitoring into active containment and risk reduction. Overall, even a basic honeypot can be a valuable component in a layered defense strategy, providing real time visibility into potential threats and helping shape informed security policies.